TECH I CAN

Kairo

Curious today, Confident tomorrow

A practical beginner guide to tiny language models

Classroom Edition (2026)

Tech I Can Series

By Paul McMurray

Build. Train. Compare. Explain.

Tech I Can
Kairo

Curious today, Confident tomorrow

Kairo: A Beginner-Friendly Guide to Building and Understanding Tiny Language Models

Paul McMurray

Classroom Edition (2026)

Dedicated to all of the budding techies of the future.
The world is ready for your brilliance.

Copyright

Copyright © 2026 Tech I Can. All rights reserved. No part of this publication may be reproduced or transmitted without written permission from the publisher, except for brief quotations used in educational review.

Published by Tech I Can, for practical AI literacy in schools and workshops. This classroom edition is designed for guided learning with reproducible exercises.

Publication Data

Publisher: Tech I Can

Publication date: May 2026

ISBN: Not assigned (Edition 1 school release, not for retail sale)

Edition: Classroom Edition (2026)

Trim size: A4 digital classroom edition

Tech I Can

Curious today, Confident tomorrow.

Printed in digital format for classroom distribution.

Contents

Preface

Intro into this chapter

Welcome to *Tech I Can: Kairo*. This book was written to make AI practical, teachable, and understandable for real classrooms.

What you are going to use

- a step-by-step learning routine
- short experiments with visible outcomes
- reflection prompts that turn actions into understanding

What you will learn in this chapter

- why this book exists
- how to approach it with confidence
- what kind of learning journey to expect

The work, clearly laid out

1. understand the purpose of the book
2. adopt the evidence-first learning mindset
3. begin with curiosity, not pressure

Examples of what you might see

```
"I do not need to know everything first."
"I can run one step, observe, and explain what changed."
```

Some explanation

❖ Lightbulb Takeaway: Confidence grows when you can explain one real change at a time, in your own words.

This guide is intentionally written for people who are curious but may be new to programming or AI. You do not need to be an expert to begin. You only need a clear process, honest outputs, and a willingness to test your assumptions.

After you interact: What you learned

- You learned that this book teaches through small, repeatable evidence cycles.
- You learned that reflection is part of the workflow, not an extra activity.
- You learned that confidence grows when you can explain one clear change at a time.

Reflection Questions

- Which part of the evidence-first approach feels most different from how you usually learn?
- What kind of learning note will help you explain your progress clearly?
- Where do you expect to need the most support as you begin?

What to Try Next in This Chapter

- Write a short learner promise that starts with: "In this book, I will test ideas by..."
- Share that promise with a partner and compare how each of you plans to track evidence.

How to Use This Book

Intro into this chapter

This quick-start page explains how to navigate the book so each chapter feels manageable and purposeful.

What you are going to use

- chapter flow pattern
- guided labs
- workbook and capstone sections

What you will learn in this chapter

- how to move through the book efficiently
- how to collect evidence while you learn
- how to choose a pacing mode

The work, clearly laid out

1. choose your pacing path (fast, standard, extended)
2. run one chapter at a time with notes
3. save outputs and reflections each session
4. revisit glossary whenever a term is unclear

Examples of what you might see

```
Fast path (2 sessions): Chapters 4-8, 13, 23-25
Standard path (4 sessions): Chapters 1-14 plus labs
Extended path (6+ sessions): full book + capstone
```

Some explanation

■ Note: Every chapter follows the same structure so learners always know what is coming next.

✦ Lightbulb Takeaway: Consistent workflow reduces overwhelm and improves confidence.

After you interact: What you learned

- You learned how the pacing paths map to real session time and class constraints.
- You learned how to move through chapters without skipping the evidence steps.
- You learned how to use the glossary and labs as support tools when a topic feels heavy.

Reflection Questions

- Which pacing path matches your current schedule, and why?
- Which chapter group should you treat as essential for your first run?
- When will you pause to record evidence so you can compare later?

What to Try Next in This Chapter

- Build a one-page reading plan for your next two sessions.
- Compare your plan with a peer and agree one shared checkpoint after each session.

Chapter 1: Welcome to Tech I Can

Intro into this chapter

You are about to learn AI by doing. This chapter sets the tone so you know what you are building and why each step matters.

What you are going to use

- this guide
- your local Kairo project
- a notebook for observations

What you will learn in this chapter

- the learning method used throughout the book
- how chapters are structured
- how to read output with confidence

The work, clearly laid out

1. learn the five-step method
2. understand chapter flow
3. prepare for practical work

Examples of what you might see

```
Run -> Observe -> Compare -> Explain -> Improve
```

Some explanation

✦ Lightbulb Takeaway: You are not here to memorize commands. You are here to build an explanation habit.

After you interact: What you learned

- You learned the five-step learning loop: run, observe, compare, explain, improve.

- You learned that this book rewards evidence, not guessing.
- You learned how to keep a simple record of what changed and why.

Reflection Questions

- Which step of the five-step loop felt easiest, and which felt hardest?
- What would count as strong evidence in this book?
- How will you capture your observations so you can compare them later?

What to Try Next in This Chapter

- Write one sentence for each step of the five-step loop using your own words.
- Create a one-page notes template with: command, output, change, explanation.

Chapter 2: What Kairo Is (and Is Not)

Intro into this chapter

Before running code, you need realistic expectations.

What you are going to use

- project overview
- sample dataset descriptions
- your own reasoning

What you will learn in this chapter

- what Kairo is designed for
- what Kairo is not trying to be
- why tiny models are great for learning

The work, clearly laid out

1. identify Kairo strengths
2. identify Kairo limits
3. connect both to classroom value

Examples of what you might see

```
Strong: clear style change after retraining
Limit: occasional repetitive text
```

Some explanation

◆ Definition: Tiny model means a model intentionally small enough to train quickly and inspect locally.

■ Note: Smaller models can fail more visibly, which is useful when teaching.

✦ Lightbulb Takeaway: Honest limitations make better learning outcomes.

After you interact: What you learned

- You learned what Kairo is built to do: show model behavior clearly in a small, teachable setup.
- You learned what Kairo is not built to do: act like a production-grade assistant.
- You learned why visible model limits are useful for real AI literacy.

Reflection Questions

- Which Kairo strength is most useful for your classroom and why?
- Which limit could confuse beginners if it is not explained early?
- How would you explain "small model, clear behavior" to a new learner?

What to Try Next in This Chapter

- Write two claims: one accurate claim about Kairo, one overclaim. Then correct the overclaim.
- Make a short "what this project can and cannot do" slide for learners.

Chapter 3: Before You Start

Intro into this chapter

This chapter prevents setup issues before they interrupt learning.

What you are going to use

- Python 3.11+
- terminal access
- local repository copy

What you will learn in this chapter

- required versus optional components
- which extras unlock classroom features
- how to verify readiness

The work, clearly laid out

1. confirm Python version
2. confirm local repository
3. decide which extras to install

Examples of what you might see

```
Required: Python 3.11+
Optional extras: learn, pdf, dev
```

Some explanation

■ **Note: Setup quality directly affects the quality of your first model run.**

✦ **Lightbulb Takeaway: A clean start removes most beginner friction.**

After you interact: What you learned

- You learned the difference between required tools and optional extras.
- You learned how early setup checks prevent later training failures.
- You learned which extras support classroom delivery and printable materials.

Reflection Questions

- Which requirement is non-negotiable before running any chapter commands?
- Which optional extra is most important for your current teaching goal?
- What is the first sign that your environment setup is incomplete?

What to Try Next in This Chapter

- Run a final preflight checklist and tick each item off by hand.
- Ask a peer to review your setup list and see if anything is missing.

Chapter 4: Install and Verify

Intro into this chapter

Now you turn this project into a working local AI lab.

What you are going to use

- virtual environment tools
- package installer
- quality checks

What you will learn in this chapter

- how to install safely
- how to verify readiness
- how to spot setup problems early

The work, clearly laid out

1. create environment
2. install project
3. run health checks

■ Snippet Purpose: Create an isolated Python environment for this project.

■ Snippet Change: This creates and activates a project-specific Python runtime so package versions do not conflict with other projects.

```
python -m venv .venv
source .venv/bin/activate
```

■ Snippet Purpose: Install Kairo and optional classroom features.

■ Snippet Change: This installs command-line tools and optional Learn/PDF/dev extras used in later chapters.

```
pip install -e .
pip install -e ".[learn]"
pip install -e ".[pdf]"
```

```
pip install -e ".[dev]"
```

■ **Snippet Purpose: Confirm code quality, compilation health, and test stability.**

■ **Snippet Change: This validates that the environment and repository are healthy before model training.**

```
ruff check .
python -m compileall src tests tools
pytest -q
```

Examples of what you might see

```
All checks passed!
122 passed in 42.53s
```

Some explanation

◆ **Definition: Virtual environment (`.venv`) is a local Python space that keeps project dependencies isolated.**

⌖ **Lightbulb Takeaway: Verification is not extra work. It is what makes your next chapters trustworthy.**

After you interact: What you learned

- You learned how to isolate project dependencies with a virtual environment.
- You learned how to install core and optional features in a controlled order.
- You learned how lint, compile, and test checks confirm that the project is healthy.

Reflection Questions

- Why is a virtual environment safer than using global Python packages?
- Which verification command gives you the fastest signal when something is wrong?
- If one check fails, what is your next troubleshooting move?

What to Try Next in This Chapter

- Deactivate and reactivate the environment, then rerun the three health checks and compare the output.
- Ask a peer to run the same checks on their machine and compare whether any results differ.

Chapter 5: Run Kairo End-to-End

Intro into this chapter

This chapter is your first complete model cycle: baseline, evaluation, retrain, comparison.

What you are going to use

- `data/samples/space_adventure.txt`
- `data/samples/pirate_dialogue.txt`
- `data/samples/README.md`
- training, generation, and evaluation commands

What you will learn in this chapter

- how to run a complete experiment
- how to compare before/after behavior fairly
- why training data changes style

The work, clearly laid out

1. train baseline model
2. generate baseline output
3. evaluate baseline quality
4. retrain on contrast dataset
5. generate again with same prompt

■ **Snippet Purpose: Train a baseline checkpoint on neutral narrative text.**

■ **Snippet Change: This creates your first reference checkpoint and establishes a baseline writing style.**

```
kairo-train --input_file data/samples/space_adventure.txt --out_dir
  runs/book_demo_normal --epochs 1 --batch_size 4 --seq_len 32 --d_model 64
  --n_heads 4 --n_layers 2 --device cpu
```

■ **Snippet Purpose: Generate baseline output from the saved checkpoint.**

■ **Snippet Change: This produces the first output sample you will later compare after retraining.**

```
kairo-generate --checkpoint runs/book_demo_normal/best.pt --prompt
  "Captain Rowan looked at the stars" --max_new_tokens 40 --temperature 0.9
  --top_k 20 --device cpu
```

■ **Snippet Purpose: Measure baseline loss and perplexity on the same dataset.**

■ **Snippet Change: This records baseline quality metrics for later before/after comparison.**

```
kairo-evaluate --checkpoint runs/book_demo_normal/best.pt --input_file
  data/samples/space_adventure.txt --device cpu
```

■ **Snippet Purpose: Retrain the same architecture on pirate dialogue for contrast.**

■ **Snippet Change: This shifts style behavior while keeping architecture fixed, enabling a fair dataset-effect test.**

```
kairo-train --input_file data/samples/pirate_dialogue.txt --out_dir
  runs/book_demo_pirate --epochs 1 --batch_size 4 --seq_len 32 --d_model 64
  --n_heads 4 --n_layers 2 --device cpu
```

■ **Snippet Purpose: Generate with the same prompt to make comparison fair.**

■ **Snippet Change: This isolates the impact of retraining by keeping prompt and generation settings unchanged.**

```
kairo-generate --checkpoint runs/book_demo_pirate/best.pt --prompt
  "Captain Rowan looked at the stars" --max_new_tokens 40 --temperature 0.9
  --top_k 20 --device cpu
```

Examples of what you might see

```
Baseline output: calm space narrative voice.
Retrained output: pirate vocabulary, exclamations, dialogue rhythm.
```

Some explanation

■ **Note: Keep prompt and architecture fixed when comparing datasets.**

✦ **Lightbulb Takeaway: Fair comparison means one major change at a time.**

After you interact: What you learned

- You learned how to run a complete baseline -> evaluate -> retrain -> compare cycle.
- You learned why keeping prompt and model settings fixed makes comparisons fair.
- You learned how dataset changes can shift vocabulary, tone, and rhythm.

Reflection Questions

- Which single variable changed between your baseline and retrained runs?
- What output evidence shows a style shift most clearly?
- Which metric did you record, and how did it support your conclusion?

What to Try Next in This Chapter

- Repeat the full cycle with a new fixed prompt and compare both result sets.
- Run one extra generation per checkpoint to check consistency.

Chapter 6: How to Read Results

Intro into this chapter

Now you move from running commands to interpreting evidence.

What you are going to use

- generated outputs
- loss/perplexity values
- comparison notes

What you will learn in this chapter

- how to assess style shifts
- how to read metrics responsibly
- how to avoid overclaiming

The work, clearly laid out

1. compare vocabulary and tone
2. compare repetition patterns
3. review metrics in context
4. write evidence-based conclusion

Examples of what you might see

```
Normal model: calmer narrative voice.
Pirate model: pirate terms and dramatic punctuation.
```

Some explanation

◆ Definition: Perplexity is a measure of uncertainty; lower usually means better dataset fit, not better universal truth.

✦ Lightbulb Takeaway: Fluent output can still be wrong. Always separate style from reliability.

After you interact: What you learned

- You learned to separate writing style changes from factual reliability.
- You learned how to read loss and perplexity as signals, not absolute truth.
- You learned how to make claims that are supported by visible output evidence.

Reflection Questions

- Which part of the output comparison is strongest evidence of style change?
- What can a lower perplexity score tell you, and what can it not tell you?
- Where could a confident-sounding output still be wrong?

What to Try Next in This Chapter

- Score two outputs with a simple rubric: tone, clarity, repetition, relevance.
- Write one careful conclusion and one overclaim, then revise the overclaim.

Chapter 7: Question-Answer Mode

Intro into this chapter

This chapter changes behavior from open text generation to QA-style responses.

What you are going to use

- `data/samples/qa_space_facts.jsonl`
- corpus-building command
- QA training and QA inference commands

What you will learn in this chapter

- how QA corpora are built
- how context anchors answers
- how fallback improves classroom reliability

The work, clearly laid out

1. build QA training corpus
2. train QA checkpoint
3. run a grounded QA query

■ Snippet Purpose: Convert structured JSONL Q/A records into training text.

■ Snippet Change: This transforms question-answer records into model-trainable plain text format.

```
kairo-build-qa-corpus --input_jsonl qa_space_facts.jsonl --output_file
  runs/qa_space_facts.txt
```

■ Snippet Purpose: Train a QA-focused checkpoint on the generated corpus.

■ Snippet Change: This creates a specialized checkpoint optimized for QA-style prompts.

```
kairo-train --input_file runs/qa_space_facts.txt --out_dir
  runs/book_qa_demo --epochs 1 --batch_size 4 --seq_len 32 --d_model 64
  --n_heads 4 --n_layers 2 --device cpu
```

■ **Snippet Purpose: Ask a question with context so answers stay grounded.**

■ **Snippet Change: This tests grounded answering behavior using explicit context support.**

```
kairo-qa --checkpoint runs/book_qa_demo/best.pt --question "Who pilots the
  Aurora?" --context "Captain Rowan is the pilot of the starship Aurora."
```

Examples of what you might see

```
Answer: Captain Rowan is the pilot of the starship Aurora.
```

Some explanation

◆ **Definition: Grounding means constraining answers to supplied context.**

⌖ **Lightbulb Takeaway: Context is your anchor when model confidence and correctness do not match.**

After you interact: What you learned

- You learned how JSONL question-answer data becomes trainable text.
- You learned how grounding (answering from provided context) improves answer stability.
- You learned why fallback behavior matters when context is missing or weak.

Reflection Questions

- Which part of your context sentence most improved answer precision?
- What output signs tell you the model answered beyond the provided context?
- Which fallback response should appear when context does not contain the answer?

What to Try Next in This Chapter

- Ask the same question with and without context, then compare answer reliability.
- Create one ambiguous question and rewrite it so grounding is easier.

Chapter 8: Learn Mode

Intro into this chapter

This chapter opens the model internals in a visual way.

What you are going to use

- `kairo-learn`
- token preview view
- probabilities and attention views

What you will learn in this chapter

- how tokenization appears visually
- how next-token probabilities guide output
- how to inspect attention patterns carefully

The work, clearly laid out

1. launch Learn Mode
2. run a short training cycle
3. compare before/after outputs
4. inspect probabilities and attention

■ Snippet Purpose: Start the Learn Mode app in your local browser.

```
kairo-learn
```

Examples of what you might see

```
Launching Kairo Learn Mode...
Local URL: http://localhost:8501
```

Some explanation

■ Note: Attention maps are interpretability aids, not proof of reasoning.

✦ Lightbulb Takeaway: Visual evidence helps learners connect abstract ideas to observable behavior.

After you interact: What you learned

- You learned how token views show what the model actually reads.
- You learned how next-token probabilities shape output choices.
- You learned how attention visuals can help discussion, but do not prove reasoning.

Reflection Questions

- What did the token view reveal that plain text output did not?
- How did changing probability settings affect output variety?
- What is one useful insight from an attention view, and one thing it cannot prove?

What to Try Next in This Chapter

- Enter three short prompts with different punctuation and compare token splits.
- Capture one screenshot each for tokens, probabilities, and attention, then annotate them.

Chapter 9: Classroom Delivery Plan

Intro into this chapter

Now you shift from learner mode to facilitator mode.

What you are going to use

- baseline/retrain workflow
- comparison prompt set
- structured reflection questions

What you will learn in this chapter

- how to run a 45-minute session
- how to guide evidence discussion
- how to connect tasks to outcomes

The work, clearly laid out

1. prediction warm-up
2. baseline run
3. retrain run
4. same-prompt comparison
5. reflection and debrief

Examples of what you might see

```
"What changed, and what evidence supports that?"
"What stayed the same because architecture stayed the same?"
```

Some explanation

✦ Lightbulb Takeaway: The debrief is where understanding deepens.

After you interact: What you learned

- You learned how to sequence a 45-minute lesson from prediction to debrief.
- You learned how to keep discussion anchored to outputs instead of opinions.
- You learned where to spend time in class so learners finish with a clear conclusion.

Reflection Questions

- Which part of your lesson needs the most facilitation: prediction, comparison, or debrief?
- What prompt will you use to move learners from "I think" to "I can show"?
- Where might timing slip, and what step will you shorten first?

What to Try Next in This Chapter

- Draft a minute-by-minute plan for one full classroom run.
- Rehearse the plan with a colleague and compare where each of you expects learner confusion.

Chapter 10: Troubleshooting

Intro into this chapter

Every real workshop includes bumps. This chapter keeps progress moving.

What you are going to use

- error messages
- run directory checks
- quick recovery settings

What you will learn in this chapter

- how to diagnose common issues
- how to recover quickly
- how to keep learner confidence high

The work, clearly laid out

1. classify issue type
2. apply targeted fix
3. rerun minimal command

Examples of what you might see

```
Error: checkpoint not found at runs/demo/best.pt
Tip: verify training completed and path is correct.
```

Some explanation

> ■ **Note: Most issues are path, environment, or expectation mismatches.**

> ✦ **Lightbulb Takeaway: Troubleshooting is not a detour. It is part of mastery.**

After you interact: What you learned

- You learned how to sort failures into path, environment, or expectation issues.
- You learned how to run a minimal rerun that confirms whether a fix worked.
- You learned how to keep learner focus when errors appear during live delivery.

Reflection Questions

- Which error type do you expect most often in your setup, and why?
- What is your fastest check when a checkpoint file cannot be found?
- How will you explain an error to learners without raising anxiety?

What to Try Next in This Chapter

- Create a quick "first five checks" card for your own machine.
- Pair with a peer: swap one real error message each and compare your fix paths.

Chapter 11: Safety and Honest Framing

Intro into this chapter

This chapter helps you communicate results responsibly.

What you are going to use

- your outputs
- metrics
- a language checklist for claims

What you will learn in this chapter

- how to avoid overclaims
- how to express uncertainty clearly
- how to model critical thinking

The work, clearly laid out

1. review output claims
2. check claim support
3. rewrite weak claims carefully

Examples of what you might see

```
Good claim: "Style shifted after pirate retraining."
Weak claim: "The model now understands pirates."
```

Some explanation

✦ Lightbulb Takeaway: Honest framing increases trust and learning quality.

After you interact: What you learned

- You learned how to rewrite weak claims into evidence-based statements.
- You learned how to separate observed behavior from assumptions about understanding.

- You learned how careful language improves trust in classroom AI discussions.

Reflection Questions

- Which phrase in your own teaching could accidentally overclaim model ability?
- What evidence must be present before you call a result "reliable"?
- How will you model uncertainty when outputs are mixed?

What to Try Next in This Chapter

- Rewrite three model claims from your notes using safer, evidence-based wording.
- Compare rewritten claims with a colleague and agree on one shared claim checklist.

Chapter 12: Build Your Own Mini Projects

Intro into this chapter

This is where guided practice becomes independent experimentation.

What you are going to use

- two contrasting datasets
- one fixed prompt
- one fixed architecture

What you will learn in this chapter

- how to design controlled experiments
- how to compare outputs rigorously
- how to write defensible conclusions

The work, clearly laid out

1. choose datasets A and B
2. keep architecture fixed
3. keep prompt fixed
4. run both experiments
5. compare outputs and metrics
6. write conclusion with evidence

Examples of what you might see

```
Project: science explainers vs dramatic dialogue
Prompt: "The team opened the hatch"
Result: clear shift in tone and rhythm
```

Some explanation

✦ Lightbulb Takeaway: Controlled experiments teach faster than complex mixed changes.

After you interact: What you learned

- You learned how to hold architecture and prompts fixed while testing dataset effects.
- You learned how to compare outputs and metrics as one argument, not separate notes.
- You learned how to turn an idea into a small experiment with defendable conclusions.

Reflection Questions

- Which variable must stay fixed in your project so the comparison remains fair?
- What evidence will you use if output style and metric signals disagree?
- How will you decide whether your mini project succeeded?

What to Try Next in This Chapter

- Design one mini project with a clear objective, fixed prompt set, and evidence rubric.
- Exchange project designs with a partner and compare fairness risks before running.

Chapter 13: Full Command Reference

Intro into this chapter

This chapter is your quick command toolkit for live use.

What you are going to use

- core train/generate/evaluate commands
- QA commands
- interactive and printable commands

What you will learn in this chapter

- where each command fits in workflow
- how to sequence commands correctly
- how to avoid command overload

The work, clearly laid out

1. run core modeling commands
2. run QA commands
3. run tool and printable commands

■ Snippet Purpose: Train a compact baseline model.

```
kairo-train --input_file data/samples/space_adventure.txt --out_dir
  runs/demo --epochs 1 --batch_size 4 --seq_len 32 --d_model 64 --n_heads 4
  --n_layers 2 --device cpu
```

■ Snippet Purpose: Generate output from the trained checkpoint.

```
kairo-generate --checkpoint runs/demo/best.pt --prompt "The robot opened
  the door" --max_new_tokens 20 --device cpu
```

■ Snippet Purpose: Evaluate loss and perplexity on a chosen dataset.

```
kairo-evaluate --checkpoint runs/demo/best.pt --input_file
  data/samples/space_adventure.txt --device cpu
```

■ **Snippet Purpose: Start a direct interactive chat loop.**

```
kairo-chat --checkpoint runs/demo/best.pt --device cpu
```

■ **Snippet Purpose: Build a QA corpus text file from structured JSONL records.**

```
kairo-build-qa-corpus --input_jsonl qa_space_facts.jsonl --output_file
  runs/qa_space_facts.txt
```

■ **Snippet Purpose: Ask a grounded QA question against a trained QA checkpoint.**

```
kairo-qa --checkpoint runs/qa_demo/best.pt --question "Who pilots the
  Aurora?" --context "Captain Rowan is the pilot of the starship Aurora."
```

■ **Snippet Purpose: Launch interactive and printable helper tools.**

```
kairo-learn
kairo-agents-dashboard
python tools/pdf/generate_printables.py
python tools/pdf/generate_tech_i_can_book.py
```

Examples of what you might see

```
Generated docs/printable/Tech_I_Can_Kairo_Book.pdf
```

Some explanation

✦ **Lightbulb Takeaway: Commands are tools for questions, not goals by themselves.**

After you interact: What you learned

- You learned how each command maps to a specific stage of the workflow.
- You learned how to avoid random command use by chaining commands with intent.
- You learned which utility commands support teaching, QA, and printable outputs.

Reflection Questions

- Which three commands form your minimum end-to-end teaching workflow?
- Which command is easiest to misuse without context, and how will you guard against that?
- When should you run utility tools instead of core model commands?

What to Try Next in This Chapter

- Build your own one-page command cheat sheet grouped by workflow stage.
- Compare your cheat sheet with a peer and merge into one classroom version.

Chapter 14: Reflection Checkpoint

Intro into this chapter

This chapter is a checkpoint where you pause, review evidence, and prepare for the deeper technical chapters ahead.

What you are going to use

- your saved runs
- your comparison notes
- your next project idea

What you will learn in this chapter

- how to consolidate what you have learned so far
- how to carry the method into deeper chapters
- how to explain your current progress clearly to others

The work, clearly laid out

1. review strongest evidence examples
2. pick one mini-project
3. rerun the method independently

Examples of what you might see

```
Run -> Observe -> Compare -> Explain -> Improve
```

Some explanation

✦ Lightbulb Takeaway: Confidence is built by repeated clear cycles, not one big perfect run.

After you interact: What you learned

- You learned how to carry the run-observe-compare-explain-improve method into new topics.

- You learned how to choose a next project that is small enough to finish and evaluate.
- You learned how to keep momentum by reviewing saved evidence rather than relying on memory.

Reflection Questions

- Which part of the method will you keep exactly the same in your next project?
- What is one realistic project you can complete in one week?
- What evidence format will help you review progress after a month?

What to Try Next in This Chapter

- Write a one-week continuation plan with one concrete model experiment.
- Share plans with a peer and compare risk, scope, and evidence quality.

Curious today, Confident tomorrow.

Chapter 15: Dataset Design for Better Training

Intro into this chapter

You now move from using existing sample files to designing your own datasets with intent.

What you are going to use

- `data/samples/README.md`
- your own source text ideas
- a dataset design checklist

What you will learn in this chapter

- how to choose data that teaches clearly
- how to avoid mixed-style confusion
- how to build classroom-friendly corpora

The work, clearly laid out

1. pick one learning objective
2. choose text that matches that objective
3. remove noisy or unrelated sections
4. test one short training run
5. inspect whether output matches intent

Examples of what you might see

```
Objective: teach style transfer
Dataset A: calm narrative
Dataset B: dramatic dialogue
Result: visible vocabulary and rhythm shift
```

Some explanation

◆ Definition: Dataset curation means choosing and shaping source text so training behavior is purposeful.

■ Note: The best learning datasets are clean, focused, and interpretable.

✦ Lightbulb Takeaway: If your dataset objective is unclear, your output analysis will also be unclear.

After you interact: What you learned

- You learned how to align dataset content with a single teaching objective.
- You learned how noisy or mixed text weakens output interpretation.
- You learned how clean corpus boundaries make before/after comparisons clearer.

Reflection Questions

- Which lines in your source text support your objective, and which should be removed?
- How would mixed tone in one dataset distort your experiment?
- What simple rule will you use to accept or reject new source text?

What to Try Next in This Chapter

- Curate a 200-400 line sample dataset focused on one voice or task.
- Swap datasets with a peer and compare whether each one has a clear objective.

Chapter 16: Prompt Design for Fair Comparisons

Intro into this chapter

Prompt design controls the fairness of your experiment.

What you are going to use

- one fixed prompt set
- your baseline and retrained checkpoints
- a prompt tracking table

What you will learn in this chapter

- how to keep prompts comparable
- how prompt wording changes output
- how to separate prompt effects from data effects

The work, clearly laid out

1. write 5 fixed prompts
2. run each prompt on baseline checkpoint
3. run same prompts on retrained checkpoint
4. compare output differences line by line

■ Snippet Purpose: Use one prompt for simple baseline comparison.

```
kairo-generate --checkpoint runs/book_demo_normal/best.pt --prompt
  "Captain Rowan looked at the stars" --max_new_tokens 40 --temperature 0.9
  --top_k 20 --device cpu
```

■ Snippet Purpose: Reuse the same prompt on the retrained model.

```
kairo-generate --checkpoint runs/book_demo_pirate/best.pt --prompt
  "Captain Rowan looked at the stars" --max_new_tokens 40 --temperature 0.9
  --top_k 20 --device cpu
```

Examples of what you might see

```
Same prompt, different style cues:
- baseline: descriptive and calm
- retrained: pirate slang and emphatic punctuation
```

Some explanation

◆ Definition: Controlled prompting means keeping prompt wording constant while changing only the condition being tested.

✦ Lightbulb Takeaway: Fixed prompts create fair evidence.

After you interact: What you learned

- You learned how fixed prompts prevent accidental drift between model comparisons.
- You learned how small wording shifts can change output style and confidence.
- You learned how prompt tracking protects fairness when sharing results.

Reflection Questions

- Which prompt words are most likely to bias model tone in your tests?
- How would you prove that a change came from data, not prompt phrasing?
- What should your prompt tracking table include to support reproducibility?

What to Try Next in This Chapter

- Build a five-prompt comparison set with clear intent labels for each prompt.
- Run the same prompt set with a peer and compare where your interpretations differ.

Chapter 17: Tokenization Deep Dive

Intro into this chapter

This chapter explains how raw text becomes model-readable tokens.

What you are going to use

- Learn Mode token preview
- short custom sentences
- punctuation-rich examples

What you will learn in this chapter

- what byte-level tokenization does
- why spacing and punctuation matter
- how token boundaries influence generation

The work, clearly laid out

1. open Learn Mode token preview
2. test simple and complex sentences
3. inspect how punctuation splits
4. compare token counts by sentence style

Examples of what you might see

```
Input: "Hello, class!"
Token pattern: H e l l o , [space] c l a s s !
```

Some explanation

◆ Definition: Byte-level tokenization represents text as byte units rather than word pieces.

■ Note: Tiny shifts in punctuation can change token sequence and model behavior.

✦ Lightbulb Takeaway: Tokens are the model's actual reading units.

After you interact: What you learned

- You learned how byte-level tokenization breaks text into units the model can process.
- You learned how punctuation and spacing choices alter token boundaries and sequence length.
- You learned how token patterns influence the style and stability of generated output.

Reflection Questions

- Which sample sentence produced more tokens than you expected, and why?
- How did punctuation change token boundaries in your Learn Mode tests?
- Which tokenization pattern best explains one output behavior you observed?

What to Try Next in This Chapter

- Build a small table of five sentences with their token counts and boundary notes.
- Swap your table with a partner and compare one sentence where your token interpretation differed.

Chapter 18: Training Dynamics and Curves

Intro into this chapter

You now learn how to read training behavior over time.

What you are going to use

- train/validation loss curves
- short and longer training runs
- comparison notes

What you will learn in this chapter

- how loss usually changes per epoch
- what overfitting can look like
- when to stop and inspect instead of continuing

The work, clearly laid out

1. run a short training session
2. record train and validation loss
3. run a slightly longer session
4. compare curve behavior

■ Snippet Purpose: Run a short training for curve observation.

```
kairo-train --input_file data/samples/space_adventure.txt --out_dir
  runs/curve_short --epochs 1 --batch_size 4 --seq_len 32 --d_model 64
  --n_heads 4 --n_layers 2 --device cpu
```

■ Snippet Purpose: Run a slightly longer training for comparison.

```
kairo-train --input_file data/samples/space_adventure.txt --out_dir
  runs/curve_long --epochs 3 --batch_size 4 --seq_len 32 --d_model 64
  --n_heads 4 --n_layers 2 --device cpu
```

Examples of what you might see

```
Epoch 1: train_loss=4.20 | val_loss=4.08
Epoch 2: train_loss=3.95 | val_loss=4.02
Epoch 3: train_loss=3.76 | val_loss=4.10
```

Some explanation

◆ Definition: Overfitting is when training performance keeps improving while generalization quality stalls or worsens.

✦ Lightbulb Takeaway: Better training loss alone is not the full story.

After you interact: What you learned

- You learned how to read train and validation loss together instead of focusing on one line.
- You learned how to spot early overfitting by watching validation stall while training keeps improving.
- You learned how to use short and longer runs (`runs/curve_short` and `runs/curve_long`) to make stop-or-continue decisions.

Reflection Questions

- At what point did validation stop improving in your longer run logs?
- Which curve pattern would make you stop training and inspect settings?
- What tradeoff did you notice between fewer epochs and output quality?

What to Try Next in This Chapter

- Run a two-epoch version of the same experiment and compare its curve shape with one and three epochs.
- Compare your curve notes with a peer and agree on one shared "stop training" rule.

Chapter 19: Evaluation Beyond One Number

Intro into this chapter

This chapter expands evaluation from single metrics to richer interpretation.

What you are going to use

- evaluation outputs
- paired prompt generations
- a rubric for quality dimensions

What you will learn in this chapter

- how to combine metrics with text evidence
- how to assess coherence and stability
- how to write balanced conclusions

The work, clearly laid out

1. run `kairo-evaluate` on two checkpoints
2. generate 5 prompt outputs from each
3. score coherence, repetition, and relevance
4. summarize with metric + output evidence

■ Snippet Purpose: Evaluate baseline checkpoint on baseline dataset.

```
kairo-evaluate --checkpoint runs/book_demo_normal/best.pt --input_file
  data/samples/space_adventure.txt --device cpu
```

■ Snippet Purpose: Evaluate retrained checkpoint on its dataset.

```
kairo-evaluate --checkpoint runs/book_demo_pirate/best.pt --input_file
  data/samples/pirate_dialogue.txt --device cpu
```

Examples of what you might see

```
Baseline: lower repetition, softer tone.
Retrained: stronger theme markers, occasional looping phrases.
```

Some explanation

■ **Note: Metrics answer "how predictable," not "how meaningful."**

✦ **Lightbulb Takeaway: Strong evaluation always combines numbers and language.**

After you interact: What you learned

- You learned how to combine `kairo-evaluate` metrics with direct text evidence from generated outputs.
- You learned how to score coherence, repetition, and relevance as separate quality dimensions.
- You learned how to write balanced evaluation statements when metrics and output quality do not fully agree.

Reflection Questions

- Which quality dimension changed most between baseline and retrained outputs?
- Which metric helped your conclusion, and where did text evidence matter more?
- How did you handle a case where a strong metric still produced weak language quality?

What to Try Next in This Chapter

- Create a one-page scoring sheet and evaluate two new prompts with the same rubric.
- Exchange scoring sheets with a partner and compare where your judgments diverged.

Chapter 20: Reliability Patterns in Tiny Models

Intro into this chapter

This chapter helps you identify common reliability patterns.

What you are going to use

- repeated prompt trials
- temperature variations
- context-grounded prompts

What you will learn in this chapter

- where tiny models are stable
- where tiny models drift
- how settings influence reliability

The work, clearly laid out

1. run same prompt 5 times
2. compare stability across outputs
3. lower and raise temperature
4. repeat with explicit context

■ Snippet Purpose: Generate with moderate randomness.

```
kairo-generate --checkpoint runs/book_demo_normal/best.pt --prompt "The
  rescue crew entered the station" --max_new_tokens 50 --temperature 0.9
  --top_k 20 --device cpu
```

■ Snippet Purpose: Generate with lower randomness for stability check.

```
kairo-generate --checkpoint runs/book_demo_normal/best.pt --prompt "The
  rescue crew entered the station" --max_new_tokens 50 --temperature 0.6
  --top_k 20 --device cpu
```

Examples of what you might see

```
temperature=0.9 -> more variation and occasional odd phrases
temperature=0.6 -> more consistent but less expressive phrasing
```

Some explanation

◆ Definition: Reliability here means consistency and groundedness across repeated runs under similar conditions.

❖ Lightbulb Takeaway: Settings can trade creativity for stability.

After you interact: What you learned

- You learned how repeated prompt trials reveal whether a checkpoint is consistent or drifting.
- You learned how lowering temperature improved stability while reducing expressive variation.
- You learned how to select generation settings that are clearer and safer for classroom demonstrations.

Reflection Questions

- What reliability difference did you observe between `temperature=0.9` and `temperature=0.6`?
- Which output traits became more stable when randomness was reduced?
- What setting choice would you use for a beginner-facing live demo, and why?

What to Try Next in This Chapter

- Run the same prompt five times at one additional temperature and log variation patterns.
- Compare your reliability logs with a peer and agree on a default classroom setting profile.

Chapter 21: Classroom Safety Workflow

Intro into this chapter

This chapter operationalizes safety for classroom demonstrations.

What you are going to use

- prompt checklist
- context-first QA prompts
- review language for student discussions

What you will learn in this chapter

- how to prevent unsafe overclaiming
- how to model responsible critique
- how to frame uncertainty constructively

The work, clearly laid out

1. define claim boundaries before demo
2. use context for factual questions
3. highlight uncertainty language in output
4. debrief limits after each run

Examples of what you might see

```
"This output is plausible but not verified."
"Let's find supporting evidence before we accept this claim."
```

Some explanation

■ **Note: Safety in this context includes epistemic safety: avoiding false certainty.**

✦ **Lightbulb Takeaway: Responsible framing is a teaching skill, not a disclaimer.**

After you interact: What you learned

- You learned how to embed safety language into each stage of a live classroom workflow.
- You learned how to model critical questioning without dismissing learner curiosity.
- You learned how to frame uncertain outputs in ways that keep discussion calm and evidence-based.

Reflection Questions

- Which classroom sentence best signals uncertainty without shutting discussion down?
- Where in your lesson flow should you add a deliberate safety check?
- Which type of learner response needs the most careful facilitation during AI output review?

What to Try Next in This Chapter

- Write a short safety script with three lines you will use during live output review.
- Rehearse the script with a colleague and compare tone, clarity, and learner impact.

Chapter 22: QA System Design Principles

Intro into this chapter

Now you treat QA mode as a system, not just a command.

What you are going to use

- corpus build step
- QA checkpoint
- context quality checklist

What you will learn in this chapter

- what makes QA training data effective
- how context quality shapes answer quality
- how fallback behavior supports reliability

The work, clearly laid out

1. inspect QA record clarity
2. build corpus and train QA checkpoint
3. test varied question forms
4. compare context-rich vs context-poor behavior

Examples of what you might see

```
Context-rich question -> specific answer
Context-poor question -> generic or drifting answer
```

Some explanation

◆ Definition: QA system quality depends on both model behavior and context design.

✦ Lightbulb Takeaway: Better context produces better answers more reliably than random parameter tweaking.

After you interact: What you learned

- You learned how QA quality depends on the combined design of corpus, checkpoint, and context.
- You learned how context-rich prompts produce more specific and trustworthy answers than weak context.
- You learned how fallback behavior design improves reliability when question context is incomplete.

Reflection Questions

- Which context quality issue caused the weakest answer in your tests?
- What corpus design choice most improved QA answer consistency?
- Which fallback behavior should trigger when context and question do not align?

What to Try Next in This Chapter

- Build a context quality checklist and test it against three new QA prompts.
- Compare checklist results with a peer and refine one shared QA design standard.

Chapter 23: Guided Lab 1 (Baseline Build)

Intro into this chapter

This lab is a full baseline run with checkpoint handling and evidence logging.

What you are going to use

- `space_adventure.txt`
- training and generation commands
- a lab notes template

What you will learn in this chapter

- how to execute a clean baseline lab
- how to capture reproducible observations
- how to store outputs for later comparison

The work, clearly laid out

1. create run directory
2. train baseline
3. generate 3 outputs from fixed prompts
4. evaluate checkpoint
5. record findings

■ Snippet Purpose: Train baseline model into a dedicated run folder.

```
kairo-train --input_file data/samples/space_adventure.txt --out_dir
  runs/lab1_baseline --epochs 1 --batch_size 4 --seq_len 32 --d_model 64
  --n_heads 4 --n_layers 2 --device cpu
```

■ Snippet Purpose: Generate output sample 1 for baseline notebook.

```
kairo-generate --checkpoint runs/lab1_baseline/best.pt --prompt "The
  mission clock started at dawn" --max_new_tokens 50 --temperature 0.8
  --top_k 20 --device cpu
```

■ **Snippet Purpose: Generate output sample 2 for baseline notebook.**

```
kairo-generate --checkpoint runs/lab1_baseline/best.pt --prompt "The
  engineers checked the hull seals" --max_new_tokens 50 --temperature 0.8
  --top_k 20 --device cpu
```

■ **Snippet Purpose: Generate output sample 3 for baseline notebook.**

```
kairo-generate --checkpoint runs/lab1_baseline/best.pt --prompt "Captain
  Rowan opened the flight log" --max_new_tokens 50 --temperature 0.8 --top_k
  20 --device cpu
```

■ **Snippet Purpose: Evaluate baseline checkpoint and capture metrics.**

```
kairo-evaluate --checkpoint runs/lab1_baseline/best.pt --input_file
  data/samples/space_adventure.txt --device cpu
```

Examples of what you might see

```
Sample outputs are coherent but sometimes repetitive.
loss: 4.0x
perplexity: mid-50s
```

Some explanation

■ **Note: Baseline labs should be easy to repeat exactly.**

⌖ **Lightbulb Takeaway: Reproducibility is a core classroom skill.**

After you interact: What you learned

- You learned how to run a clean baseline workflow from training to evaluation in `runs/lab1_baseline`.
- You learned how to document three fixed-prompt generations so later comparisons remain fair.
- You learned how to capture baseline metrics and notes that prepare a valid retrain contrast in Lab 2.

Reflection Questions

- Which of your three baseline prompts produced the most repeatable output pattern?
- What baseline metric will be most useful when you compare against Lab 2 results?

- Which note format made it easiest to connect output text with evaluation numbers?

What to Try Next in This Chapter

- Add one new fixed baseline prompt and record output with the same settings used in this lab.
- Compare baseline logs with a peer and identify one documentation improvement you both will adopt.

Chapter 24: Guided Lab 2 (Retrain Contrast)

Intro into this chapter

This lab demonstrates clear style transfer through retraining.

What you are going to use

- `pirate_dialogue.txt`
- same architecture and prompts from Lab 1
- comparison worksheet

What you will learn in this chapter

- how retraining shifts style
- how to preserve fair comparisons
- how to document changes with evidence

The work, clearly laid out

1. retrain model with pirate data
2. rerun same 3 prompts
3. compare vocabulary, tone, and structure
4. write evidence-based summary

■ Snippet Purpose: Train contrast model with same architecture settings.

```
kairo-train --input_file data/samples/pirate_dialogue.txt --out_dir
  runs/lab2_pirate --epochs 1 --batch_size 4 --seq_len 32 --d_model 64
  --n_heads 4 --n_layers 2 --device cpu
```

■ Snippet Purpose: Re-run prompt 1 against retrained checkpoint.

```
kairo-generate --checkpoint runs/lab2_pirate/best.pt --prompt "The mission
  clock started at dawn" --max_new_tokens 50 --temperature 0.8 --top_k 20
  --device cpu
```

■ Snippet Purpose: Re-run prompt 2 against retrained checkpoint.

```
kairo-generate --checkpoint runs/lab2_pirate/best.pt --prompt "The
  engineers checked the hull seals" --max_new_tokens 50 --temperature 0.8
  --top_k 20 --device cpu
```

■ Snippet Purpose: Re-run prompt 3 against retrained checkpoint.

```
kairo-generate --checkpoint runs/lab2_pirate/best.pt --prompt "Captain
  Rowan opened the flight log" --max_new_tokens 50 --temperature 0.8 --top_k
  20 --device cpu
```

Examples of what you might see

```
Pirate run uses "matey", "crew", "yo-ho", and dramatic punctuation.
Sentence rhythm shifts toward dialogue and commands.
```

Some explanation

◆ Definition: Style transfer in this context means shifting generated language patterns through changed training data.

⌖ Lightbulb Takeaway: Same model structure, different data, different behavior.

After you interact: What you learned

- You learned how retraining on `pirate_dialogue.txt` changed language style while architecture stayed fixed.
- You learned how reusing the same three prompts made the baseline versus retrain comparison defensible.
- You learned how to explain style transfer using concrete evidence in vocabulary, tone, and sentence rhythm.

Reflection Questions

- Which specific words or punctuation patterns most clearly signaled style transfer?
- Which prompt showed the strongest contrast between Lab 1 and Lab 2 outputs?
- How did fixed settings protect your conclusion from accidental bias?

What to Try Next in This Chapter

- Write a short contrast summary that cites one output line from each of the three prompts.
- Exchange summaries with a peer and compare whether both of you chose the same strongest evidence.

Chapter 25: Guided Lab 3 (QA Grounding)

Intro into this chapter

This lab turns the model into a basic context-grounded QA experience.

What you are going to use

- `qa_space_facts.jsonl`
- corpus builder
- QA command with context

What you will learn in this chapter

- how to prepare QA data
- how to test grounded answers
- how to detect drift and recovery

The work, clearly laid out

1. build QA corpus
2. train QA checkpoint
3. ask 5 factual questions
4. compare with and without context

■ Snippet Purpose: Build QA corpus text from JSONL records.

```
kairo-build-qa-corpus --input_jsonl qa_space_facts.jsonl --output_file
  runs/lab3_qa_corpus.txt
```

■ Snippet Purpose: Train QA checkpoint from lab corpus.

```
kairo-train --input_file runs/lab3_qa_corpus.txt --out_dir runs/lab3_qa
  --epochs 1 --batch_size 4 --seq_len 32 --d_model 64 --n_heads 4 --n_layers
  2 --device cpu
```

■ Snippet Purpose: Ask a grounded question with explicit context.

```
kairo-qa --checkpoint runs/lab3_qa/best.pt --question "Who pilots the
  Aurora?" --context "Captain Rowan is the pilot of the starship Aurora."
```

■ **Snippet Purpose: Ask a second grounded question for consistency check.**

```
kairo-qa --checkpoint runs/lab3_qa/best.pt --question "What powers the
  Aurora?" --context "The Aurora uses a helium-3 fusion core."
```

Examples of what you might see

```
Context provided -> concise grounded answer.
Context missing -> broader, less anchored phrasing.
```

Some explanation

■ **Note: QA quality improves most through context quality, not purely model size.**

✦ **Lightbulb Takeaway: Good context design is the strongest lever in classroom QA.**

After you interact: What you learned

- You learned how to build and train a QA checkpoint from structured facts using a repeatable lab workflow.
- You learned how answer quality improved when context lines were specific and directly relevant to the question.
- You learned how to detect drift by comparing grounded and ungrounded responses across the same question set.

Reflection Questions

- Which question in your lab produced the clearest grounded answer, and what made the context effective?
- What output clue showed the model was drifting beyond the provided facts?
- Which context edit improved a weak answer the most?

What to Try Next in This Chapter

- Run five questions with context and five without context, then score groundedness for each answer.

- Trade question-context pairs with a peer and compare where each of you judged grounding differently.

Chapter 26: Guided Lab 4 (Learn Mode Evidence Walkthrough)

Intro into this chapter

This lab helps students connect model internals to observable output behavior.

What you are going to use

- Learn Mode interface
- a fixed prompt list
- probability and attention views

What you will learn in this chapter

- how to read probability distributions
- how to interpret attention cautiously
- how to narrate internals in plain language

The work, clearly laid out

1. open Learn Mode
2. run one prompt in token view
3. inspect top next-token probabilities
4. inspect attention map for same example
5. write a two-sentence evidence summary

■ Snippet Purpose: Launch Learn Mode app for visual exploration.

```
kairo-learn
```

Examples of what you might see

```
Top token candidates:
1) "the" 0.18
2) "and" 0.14
3) "to"  0.11
```

Some explanation

◆ **Definition: Next-token probabilities are candidate likelihoods before sampling.**

■ **Note: Attention indicates influence patterns, not definitive reasoning intent.**

❖ **Lightbulb Takeaway: Visual evidence helps learners ask better questions.**

After you interact: What you learned

- You learned how to connect top-token probabilities to the actual words the model produced next.
- You learned how to discuss attention views as influence signals without claiming they prove full reasoning.
- You learned how to guide learners from visual observations to clear, evidence-based interpretations.

Reflection Questions

- Which probability shift best explained a surprising output choice in your walkthrough?
- What is one attention pattern you observed, and what is the safest conclusion you can draw from it?
- Which visual panel helped most when explaining model behavior to learners?

What to Try Next in This Chapter

- Capture one Learn Mode example and annotate tokens, probabilities, and attention in a single evidence sheet.
- Compare your annotated sheet with a peer and discuss one place where your interpretations differ.

Chapter 27: Classroom Assessment and Feedback

Intro into this chapter

This chapter provides concrete ways to assess learning outcomes.

What you are going to use

- student lab notes
- output comparison artifacts
- reflection prompts and rubric criteria

What you will learn in this chapter

- how to assess process and explanation quality
- how to grade evidence use fairly
- how to provide actionable feedback

The work, clearly laid out

1. assess workflow completion
2. assess evidence quality
3. assess explanation clarity
4. provide next-step feedback

Examples of what you might see

```
Strong response:
"I kept the prompt fixed and changed only dataset. Output tone shifted
  from calm narrative to pirate dialogue markers."
```

Some explanation

◆ Definition: Evidence quality means claims are supported by concrete output and workflow details.

✦ Lightbulb Takeaway: Grade thinking quality, not just command execution.

After you interact: What you learned

- You learned how to assess learner work by weighting evidence quality, method quality, and explanation clarity.
- You learned how to give feedback that improves reasoning, not just command execution speed.
- You learned how rubric-based assessment can guide the next experiment step for each learner.

Reflection Questions

- Which rubric category most clearly distinguished strong and weak submissions?
- What feedback sentence would help a learner improve evidence use in the next lab?
- Where did learners show understanding even when their command output was imperfect?

What to Try Next in This Chapter

- Score two sample responses using your rubric and write one concrete next-step comment for each.
- Exchange scoring with a colleague and compare how consistently you applied each rubric criterion.

Chapter 28: Presentation-Day Playbook

Intro into this chapter

This chapter prepares you for live delivery in front of students or reviewers.

What you are going to use

- one known-good checkpoint backup
- one scripted demo sequence
- one troubleshooting fallback plan

What you will learn in this chapter

- how to prepare a smooth live session
- how to handle surprises calmly
- how to keep outcomes teachable

The work, clearly laid out

1. rehearse complete demo once
2. pre-generate one backup output set
3. verify checkpoints and paths
4. pre-write key reflection questions

Examples of what you might see

```
Plan A: live train + generate
Plan B: load known-good checkpoint + run comparison prompts
```

Some explanation

■ **Note: A calm fallback plan is a mark of professional readiness.**

✦ **Lightbulb Takeaway: Great presentations are prepared for both success and hiccups.**

After you interact: What you learned

- You learned how to prepare a live demo with a primary path and a ready fallback path.
- You learned how backup checkpoints and scripted transitions reduce disruption when technical issues appear.
- You learned how to preserve teaching value by keeping comparisons and reflection prompts ready under time pressure.

Reflection Questions

- Which part of your live sequence is most likely to fail, and what is your fallback move?
- What pre-generated artifact gives you the biggest recovery advantage during a demo?
- How will you keep learners engaged if you must switch from Plan A to Plan B?

What to Try Next in This Chapter

- Rehearse your full presentation once using only Plan B resources.
- Run a peer mock session where one person introduces a surprise failure and the other recovers live.

Chapter 29: Extension Pathways

Intro into this chapter

This chapter helps you keep growing after the core curriculum.

What you are going to use

- your completed labs
- extension project ideas
- local documentation and guides

What you will learn in this chapter

- how to design advanced follow-up projects
- how to compare multiple domains
- how to turn this into ongoing coursework

The work, clearly laid out

1. select one extension theme
2. define measurable question
3. run controlled experiment
4. document findings and limits
5. present results to peers

Examples of what you might see

```
Extension themes:
- science vs poetry style transfer
- QA reliability by context quality
- prompt sensitivity by temperature range
```

Some explanation

✦ Lightbulb Takeaway: Progress comes from asking better questions, not bigger models.

After you interact: What you learned

- You learned how to scope extension projects with one measurable question and clear controls.
- You learned how to choose extension themes that build directly on your completed lab evidence.
- You learned how to sustain long-term inquiry by documenting limits as well as positive findings.

Reflection Questions

- Which extension theme gives you the strongest question for controlled testing?
- What control variables must remain fixed in your chosen extension project?
- What evidence will show that your extension result is meaningful rather than incidental?

What to Try Next in This Chapter

- Draft a one-page extension proposal with objective, controls, variables, and evidence targets.
- Review proposals with a peer and compare where each plan risks uncontrolled variables.

Chapter 30: Advanced Prompt Engineering Lab

Intro into this chapter

This chapter gives you a structured way to test how prompt design affects model behavior under fixed training conditions.

What you are going to use

- one checkpoint
- a prompt matrix
- temperature and top-k settings

What you will learn in this chapter

- how framing changes outputs
- how to build prompt families
- how to compare outputs systematically

The work, clearly laid out

1. define three prompt families (narrative, instructional, reflective)
2. generate three outputs per family
3. vary one generation parameter at a time
4. summarize output differences in a table

■ Snippet Purpose: Narrative framing prompt for baseline language style.

```
kairo-generate --checkpoint runs/lab1_baseline/best.pt --prompt "Write a
  scene where the crew discovers a hidden signal." --max_new_tokens 60
  --temperature 0.8 --top_k 20 --device cpu
```

■ Snippet Purpose: Instructional framing prompt for stepwise tone.

```
kairo-generate --checkpoint runs/lab1_baseline/best.pt --prompt "List
  three steps the crew should follow to investigate a hidden signal."
  --max_new_tokens 60 --temperature 0.8 --top_k 20 --device cpu
```

■ **Snippet Purpose: Reflective framing prompt for reasoning style language.**

```
kairo-generate --checkpoint runs/lab1_baseline/best.pt --prompt "Reflect
  on why the crew should be cautious with unknown signals." --max_new_tokens
  60 --temperature 0.8 --top_k 20 --device cpu
```

Examples of what you might see

```
Narrative prompt -> scene description and characters
Instructional prompt -> numbered or imperative style
Reflective prompt -> explanatory and opinionated language
```

Some explanation

◆ **Definition: Prompt family means a reusable pattern of instruction style.**

■ **Note: Prompt effects can be large enough to mask dataset effects if you do not control for them.**

⌖ **Lightbulb Takeaway: Prompt design is part of experimental design.**

After you interact: What you learned

- You learned how prompt family design changes output behavior even when model and data stay fixed.
- You learned how to isolate framing effects by varying one prompt or one generation setting at a time.
- You learned how to report prompt sensitivity with structured comparisons across narrative, instructional, and reflective prompts.

Reflection Questions

- Which prompt family produced the largest shift in tone or structure, and why?
- Where did parameter changes matter less than prompt framing in your results?
- What table format made prompt sensitivity easiest to compare across runs?

What to Try Next in This Chapter

- Build a nine-row prompt matrix (three families by three prompts) and run one controlled comparison pass.

- Exchange matrices with a peer and compare where framing effects were strongest and weakest.

Chapter 31: Data Ethics and Attribution

Intro into this chapter

As projects scale, source data quality and attribution become critical.

What you are going to use

- dataset origin notes
- a source tracking template
- an ethics checklist

What you will learn in this chapter

- how to document source provenance
- how to explain dataset limitations responsibly
- how to teach ethical data choices

The work, clearly laid out

1. record each dataset source
2. note intended learning objective
3. note potential bias or representation gaps
4. include data caveats in presentation

Examples of what you might see

```
Dataset: pirate_dialogue.txt
Source type: fictional script-style writing
Likely bias: exaggerated speech patterns
Teaching note: useful for style shift, not factual QA
```

Some explanation

◆ **Definition: Provenance means where data came from and how it was prepared.**

■ **Note: Good documentation improves trust and reproducibility.**

✦ Lightbulb Takeaway: Responsible AI education includes responsible data stories.

After you interact: What you learned

- You learned how provenance notes make dataset choices traceable and easier to justify.
- You learned how to describe bias and representation limits without weakening technical rigor.
- You learned how ethical attribution improves reproducibility, trust, and classroom discussion quality.

Reflection Questions

- Which provenance detail is most important when sharing a dataset with learners?
- What bias risk in your current data should be disclosed before interpretation?
- How would missing attribution reduce confidence in your findings?

What to Try Next in This Chapter

- Create a provenance card for one dataset including source type, objective, and likely bias gaps.
- Swap provenance cards with a peer and review whether each one is clear enough for classroom use.

Chapter 32: Full Classroom Script Pack

Intro into this chapter

This chapter gives complete talk tracks you can use in live teaching.

What you are going to use

- ready-to-use script blocks
- transition lines between activities
- reflection prompts

What you will learn in this chapter

- how to explain concepts in beginner language
- how to move smoothly between technical steps
- how to maintain engagement

The work, clearly laid out

1. select script set by lesson length
2. rehearse transitions
3. run one script live
4. adjust pacing based on learner response

Examples of what you might see

```
Teacher line: "We are going to keep the model architecture the same and
  change
only data. That way we can observe one clear cause of change."
```

Some explanation

◆ Definition: Script pack means pre-written classroom language aligned to workflow steps.

✦ Lightbulb Takeaway: Clear language is a technical tool.

20-minute quick script

1. "Today we are observing how data changes model behavior."
2. "Watch the baseline output first. Do not judge quality yet; just observe."
3. "Now we retrain with different text and use the same prompt."
4. "What changed? Please cite one exact phrase as evidence."
5. "What stayed similar? That likely comes from architecture and settings."
6. "Final takeaway: data influences style, but reliability still needs review."

35-minute standard script

1. "Before we run, what do you predict will change after retraining?"
2. "We train baseline and capture one output sample."
3. "Now we retrain using a contrasting dataset."
4. "Run same prompt. Compare tone, word choice, and sentence rhythm."
5. "Let's connect this to metrics: loss and perplexity."
6. "Metrics help, but output evidence explains behavior."
7. "We will now test one grounded QA question with context."
8. "Notice how context anchors answers."
9. "Exit reflection: one claim, one piece of evidence."

45-minute extended script

1. "Opening question: What does it mean for a model to learn?"
2. "Baseline training run with narration of each command purpose."
3. "Baseline generation and first evidence note."
4. "Retrain run and second evidence note."
5. "Side-by-side comparison and class discussion."
6. "Mini deep dive: token probabilities in Learn Mode."
7. "Grounded QA demonstration."
8. "Safety framing: fluent text is not automatically true."
9. "Written debrief using claim-evidence format."

Troubleshooting script lines

- "This error is normal and fixable. We will locate the checkpoint path first."
- "If training is slow, we reduce epochs and keep the learning objective."
- "Unexpected output is data for discussion, not failure."

Reflection script lines

- "What changed, and what evidence supports your claim?"
- "What uncertainty remains?"
- "What would you test next to increase confidence?"

After you interact: What you learned

- You learned how scripted transitions keep lesson flow clear between commands, comparison, and debrief.
- You learned how prewritten lines can translate technical actions into beginner-friendly classroom language.
- You learned how script prompts can keep student discussion focused on claims and evidence.

Reflection Questions

- Which script line best moved students from observation to evidence-backed claims?
- Where in your lesson did scripted transitions improve pacing the most?
- Which part of your script needs revision to sound more natural in your own voice?

What to Try Next in This Chapter

- Adapt one script set to your own classroom style and timing, then rehearse it once out loud.
- Pair with a colleague, run alternating script sections, and compare which lines produced clearer learner responses.

Chapter 33: Student Workbook Section

Intro into this chapter

This chapter provides workbook-style pages that can be used directly in class.

What you are going to use

- guided prompts
- comparison tables
- reflection checklists

What you will learn in this chapter

- how to convert model runs into learning artifacts
- how to strengthen reasoning through writing
- how to produce assessable student evidence

The work, clearly laid out

1. complete baseline worksheet
2. complete retrain worksheet
3. complete QA worksheet
4. complete reflection summary

Examples of what you might see

```
Claim: Pirate retrain changed tone.
Evidence: Output includes "matey" and command-style lines.
Confidence: Medium (style signal strong, factuality not tested).
```

Some explanation

■ **Note: Written reflection increases retention and improves discussion quality.**

✦ **Lightbulb Takeaway: Output becomes learning when learners explain it in their own words.**

Worksheet A: Baseline Observation

1. Prompt used:
2. First generated sentence:
3. Tone description:
4. Repetition observed (yes/no):
5. One question you still have:

Worksheet B: Retrain Comparison

1. Same prompt reused (yes/no):
2. New vocabulary observed:
3. Tone shift observed:
4. Structure shift observed:
5. Most convincing evidence line:

Worksheet C: Metric Reflection

1. Baseline loss:
2. Retrain loss:
3. Baseline perplexity:
4. Retrain perplexity:
5. What do metrics suggest?
6. What do metrics not prove?

Worksheet D: QA Grounding

1. Question asked:
2. Context used:
3. Answer returned:
4. Groundedness score (1-5):
5. If weak, what context change would you make?

Worksheet E: Claim-Evidence Summary

1. Claim 1:
2. Evidence for claim 1:
3. Claim 2:
4. Evidence for claim 2:
5. Uncertainty statement:
6. Next experiment step:

After you interact: What you learned

- You learned how each workbook sheet turns model output into evidence you can assess, not just observe.
- You learned how the claim-evidence and uncertainty prompts push you to justify conclusions from baseline, retrain, and QA runs.
- You learned how groundedness scoring and metric reflection help you separate style changes from reliability claims.

Reflection Questions

- Which worksheet gave you the strongest evidence for a real behavior shift, and what made that evidence strong?
- Where did your confidence score and your written uncertainty statement disagree, and why?
- In Worksheet D, what specific context edit would most improve your groundedness score?

What to Try Next in This Chapter

- Run one new prompt through Worksheets A, B, and E, then write a tighter claim using only one sentence of evidence.
- Swap completed worksheets with a partner and compare whether you both judged the same output line as the strongest evidence.

Chapter 34: Capstone Project Handbook

Intro into this chapter

This chapter helps learners build and present a complete final project.

What you are going to use

- capstone planning template
- run logs
- output comparison portfolio

What you will learn in this chapter

- how to scope a capstone well
- how to present findings professionally
- how to communicate limits honestly

The work, clearly laid out

1. define project question
2. design controlled experiment
3. run baseline and variant workflows
4. collect outputs and metrics
5. present claim-evidence conclusion

Examples of what you might see

```
Question: How does poetic training data affect response rhythm?
Method: fixed architecture + fixed prompt set + dataset swap
Finding: higher metaphor density and shorter clause structure
Limit: factual reliability not improved by style-focused data
```

Some explanation

◆ **Definition: Capstone is a culminating project demonstrating method mastery.**

✦ Lightbulb Takeaway: A strong capstone explains both what changed and what remains uncertain.

Capstone template (one-page)

1. Project title:
2. Research question:
3. Datasets used:
4. Fixed controls:
5. Variables changed:
6. Commands run:
7. Key outputs:
8. Metrics summary:
9. Main claim:
10. Supporting evidence:
11. Limitation statement:
12. Recommended next test:

Presentation rubric (10-point)

1. Clear question (2 pts)
2. Controlled method (2 pts)
3. Evidence quality (2 pts)
4. Honest limitations (2 pts)
5. Delivery clarity (2 pts)

After you interact: What you learned

- You learned how to frame a capstone question that is specific enough to test with fixed controls.
- You learned how to connect commands, outputs, and metrics into one coherent claim-evidence story.
- You learned how to report limits honestly so your project conclusions stay credible.

Reflection Questions

- Which fixed control in your capstone design protected your conclusion the most?
- Which part of your evidence chain felt weakest: outputs, metrics, or limitation statement?
- What is one claim from your capstone that you can support with two different kinds of evidence?

What to Try Next in This Chapter

- Draft a second capstone question that changes only one variable from your first plan.
- Present your one-page capstone template to a peer and compare which rubric category each of you scored highest.

Chapter 35: Implementation Workbook (Week-by-Week)

Intro into this chapter

This chapter turns the full book into a practical multi-week teaching schedule.

What you are going to use

- weekly lesson map
- checkpoint milestones
- reflection checkpoints

What you will learn in this chapter

- how to spread learning across multiple sessions
- how to track progress week by week
- how to integrate practice and reflection

The work, clearly laid out

1. choose 4-week or 6-week format
2. map chapters to weekly goals
3. define weekly evidence artifacts
4. plan mid-point and final review

Examples of what you might see

```
Week 1: setup + baseline run
Week 2: retrain + comparison
Week 3: QA mode + grounding
Week 4: capstone presentation
```

Some explanation

✦ Lightbulb Takeaway: Long-term learning works best when each week produces a small, concrete artifact.

Four-week implementation map

Week 1 objectives

- install and verify environment
- run first baseline model
- capture first output evidence

Week 1 deliverables

- successful environment check
- one baseline checkpoint
- one prompt/output reflection sheet

Week 2 objectives

- run retrain contrast workflow
- compare fixed prompt outputs
- discuss style and reliability differences

Week 2 deliverables

- retrain checkpoint
- side-by-side output comparison notes
- claim-evidence summary

Week 3 objectives

- build QA corpus and train QA checkpoint
- run grounded QA questions
- inspect Learn Mode visuals

Week 3 deliverables

- QA run log
- groundedness reflection

- one Learn Mode observation note

Week 4 objectives

- design mini-project
- run final comparison
- present findings and limitations

Week 4 deliverables

- capstone one-pager
- presentation slides or poster
- reflection on next experiment

Six-week implementation map

Week 1

- onboarding, setup, baseline run

Week 2

- retrain workflow and fair comparisons

Week 3

- deeper evaluation and metric interpretation

Week 4

- QA workflow and context engineering

Week 5

- guided lab extensions and troubleshooting

Week 6

- capstone delivery and peer review

After you interact: What you learned

- You learned how to map chapters into weekly objectives that produce visible progress artifacts.
- You learned how the four-week and six-week plans change pacing while keeping the same core learning outcomes.
- You learned how to use deliverables to catch learning gaps early instead of waiting for the final presentation.

Reflection Questions

- Which week in your chosen plan carries the highest cognitive load, and how will you reduce it?
- Which deliverable gives the clearest signal that students understood grounded QA?
- What is one milestone you would move earlier or later after reviewing your class timing?

What to Try Next in This Chapter

- Build a custom five-week schedule using the existing objectives and write one success signal for each week.
- Compare your schedule with another teacher or learner and agree on one shared checkpoint both plans should include.

Chapter 36: Frequently Asked Questions (Classroom Edition)

Intro into this chapter

This chapter provides practical answers to the questions teachers and students ask most often.

What you are going to use

- FAQ prompts
- short answer explanations
- decision guides

What you will learn in this chapter

- how to answer common conceptual questions
- how to handle confusion points quickly
- how to keep explanations grounded and consistent

The work, clearly laid out

1. review FAQs before teaching
2. pick answers that match learner level
3. use examples from your own runs

Examples of what you might see

```
Q: If loss goes down, does that mean the model is "smart"?
A: It means better fit to that dataset, not human understanding.
```

Some explanation

✦ Lightbulb Takeaway: Good answers are short, honest, and evidence-linked.

FAQ set

Q1: Why does the model repeat itself?

A: Tiny models have limited capacity and may lock into short loops, especially when training data is narrow or prompts are open-ended.

Q2: Why do we keep the prompt fixed?

A: So that output differences are more likely caused by data changes instead of prompt wording changes.

Q3: Why is context so important in QA mode?

A: Context provides grounding. Without it, the model may produce plausible but unsupported answers.

Q4: Can a low perplexity model still be wrong?

A: Yes. Perplexity indicates predictability on a dataset, not factual accuracy across all topics.

Q5: Why does a pirate-trained model affect even neutral prompts?

A: Training shifts token likelihood patterns, so style cues appear even when the prompt is neutral.

Q6: Should we train for more epochs by default?

A: Not always. For classroom goals, shorter runs are often enough to show key behavior changes without overfitting or long delays.

Q7: What if students think the model "understands"?

A: Ask them to provide evidence and distinguish fluent pattern generation from human-like understanding.

Q8: What if outputs look random?

A: Check data quality, context quality, and generation settings. Then rerun a small controlled experiment.

Q9: How do I keep lessons inclusive for mixed ability groups?

A: Use role-based tasks: some students run commands, others annotate outputs, others lead evidence review.

Q10: What is a quick success criterion for a lesson?

A: Students can state one claim and one supporting output line, plus one limitation statement.

After you interact: What you learned

- You learned how to answer recurring AI questions with short explanations tied to observed classroom behavior.
- You learned how to correct common misunderstandings, such as equating low loss with true understanding, without overcomplicating the explanation.
- You learned how to use FAQ responses to keep discussions evidence-based when students make broad claims.

Reflection Questions

- Which FAQ answer would most help if a student says, "The model is smart because it sounds fluent"?
- Which question in this chapter needs a local classroom example from your own runs to land better?
- Which two FAQ entries are most useful to pair during a live debrief, and why?

What to Try Next in This Chapter

- Write one new FAQ and answer based on a confusion point from your last session.
- In pairs, role-play a student question and teacher response, then compare which wording felt clearer and more accurate.

Chapter 37: Teacher Quick-Reference Cards

Intro into this chapter

This chapter gives compact reference cards for live teaching moments.

What you are going to use

- quick command cards
- quick explanation cards
- quick troubleshooting cards

What you will learn in this chapter

- how to respond quickly during lessons
- how to keep language concise under pressure
- how to maintain flow during demos

The work, clearly laid out

1. print or copy quick-reference cards
2. keep one card set visible during demos
3. use cards for rapid transitions and recovery

Examples of what you might see

```
Card: "Baseline Run"
Goal: produce first checkpoint and one output sample.
```

Some explanation

✦ **Lightbulb Takeaway: Fast reference tools reduce teaching cognitive load.**

Card set A: Command cards

Card A1: Baseline training

■ Snippet Purpose: Quick card for creating a baseline checkpoint during live teaching.

```
kairo-train --input_file data/samples/space_adventure.txt --out_dir
  runs/card_baseline --epochs 1 --batch_size 4 --seq_len 32 --d_model 64
  --n_heads 4 --n_layers 2 --device cpu
```

Card A2: Baseline generation

■ Snippet Purpose: Quick card for generating a baseline output sample on demand.

```
kairo-generate --checkpoint runs/card_baseline/best.pt --prompt "The
  station doors opened slowly" --max_new_tokens 40 --temperature 0.8 --top_k
  20 --device cpu
```

Card A3: Retrain contrast

■ Snippet Purpose: Quick card for training the contrast model with pirate-style data.

```
kairo-train --input_file data/samples/pirate_dialogue.txt --out_dir
  runs/card_pirate --epochs 1 --batch_size 4 --seq_len 32 --d_model 64
  --n_heads 4 --n_layers 2 --device cpu
```

Card A4: QA quick run

■ Snippet Purpose: Quick card for demonstrating context-grounded QA in class.

```
kairo-qa --checkpoint runs/lab3_qa/best.pt --question "Who pilots the
  Aurora?" --context "Captain Rowan is the pilot of the starship Aurora."
```

Card set B: Explanation cards

- "Same architecture, different data, different behavior."
- "Fluent output is not guaranteed truth."
- "Context improves grounded QA answers."
- "Metrics guide us, but output evidence explains behavior."

Card set C: Troubleshooting cards

- Missing checkpoint: verify run directory and `best.pt` path.
- Slow training: reduce epochs or sequence length.

- Noisy QA output: add clearer context.
- Unexpected style: check which dataset trained the checkpoint.

Card set D: Reflection cards

- "What changed?"
- "What evidence proves it?"
- "What remains uncertain?"
- "What would you test next?"

After you interact: What you learned

- You learned how command cards reduce demo friction by keeping key runs immediately available.
- You learned how explanation and troubleshooting cards help you recover quickly when outputs or checkpoints do not match expectations.
- You learned how reflection cards keep student discussion focused on claims, evidence, and next tests during live sessions.

Reflection Questions

- Which card from Set A would you place first in a lesson, and what transition line would you say before using it?
- Which troubleshooting card would most likely save time in your current teaching setup?
- Which reflection card prompt produces the most specific student evidence statements in your group?

What to Try Next in This Chapter

- Build a one-page quick-reference sheet by selecting only six cards you will actually use next session.
- Exchange your selected cards with a peer and compare which card each of you considers essential and why.

Chapter 38: Deployment and Operations Checklist

Intro into this chapter

This chapter helps you move from workshop experiments to repeatable operational delivery in schools or clubs.

What you are going to use

- deployment checklist
- run directory conventions
- backup and recovery plan

What you will learn in this chapter

- how to prepare a stable classroom deployment
- how to avoid common operational pitfalls
- how to recover quickly when issues appear

The work, clearly laid out

1. standardize project folder structure
2. prepare known-good checkpoints
3. prepare backup command cards
4. preflight the environment before sessions
5. capture run logs after each class

Examples of what you might see

```
Deployment status:
- environment verified
- baseline checkpoint present
- QA checkpoint present
- fallback script prepared
```

Some explanation

◆ **Definition: Operational readiness means the lesson can run reliably for learners with predictable outcomes and recoverable failure modes.**

■ **Note: Reliability is not just model quality. It is also process quality.**

⌖ **Lightbulb Takeaway: A good deployment plan saves your teaching time for learning, not debugging.**

Session preflight checklist

1. virtual environment activates successfully
2. `kairo-train --help` and `kairo-generate --help` run
3. baseline and retrain checkpoints exist
4. one prompt test runs in under expected time
5. presentation prompts and reflection questions are ready

Session postflight checklist

1. save outputs in dated folder
2. archive metrics and notes
3. record one lesson improvement for next session
4. verify cleanup and restore known-good state

Command health checks

■ **Snippet Purpose: Verify CLI commands are available before class starts.**

```
kairo-train --help
kairo-generate --help
kairo-evaluate --help
kairo-qa --help
```

■ **Snippet Purpose: Verify a known-good baseline checkpoint is loadable.**

```
kairo-generate --checkpoint runs/card_baseline/best.pt --prompt "System
  check prompt" --max_new_tokens 12 --device cpu
```

Failure mode playbook

- **Missing checkpoint**: use known-good fallback checkpoint card.
- **Slow runtime**: reduce `--max_new_tokens` and skip long retrains.
- **Environment mismatch**: re-activate `.venv` and rerun help checks.
- **Noisy QA answer**: switch to context-first QA examples.

After you interact: What you learned

- You learned how preflight checks catch missing tools and checkpoints before learners are waiting.
- You learned how postflight routines preserve outputs and notes so each class improves the next one.
- You learned how a failure-mode playbook turns common runtime issues into planned recovery steps.

Reflection Questions

- Which preflight check in this chapter would most likely prevent a full lesson delay?
- In your environment, which failure mode is highest risk and what is your first fallback action?
- Which postflight item gives you the most useful evidence for improving the next session?

What to Try Next in This Chapter

- Run a full dry-run using the checklist and record the first point where timing slips.
- Pair with a colleague to simulate one failure mode and compare your recovery paths for speed and clarity.

Chapter 39: Accessibility and Inclusive Teaching Patterns

Intro into this chapter

This chapter helps you make Kairo lessons inclusive for diverse learners.

What you are going to use

- differentiated task roles
- accessibility checkpoints
- alternative output formats

What you will learn in this chapter

- how to support mixed experience levels
- how to design equitable participation
- how to reduce cognitive overload

The work, clearly laid out

1. assign multi-role team structure
2. provide visual and text alternatives
3. scaffold reflection prompts by level
4. allow multiple evidence formats
5. debrief inclusively

Examples of what you might see

```
Role split:
- operator: runs commands
- observer: records output patterns
- analyst: writes claim-evidence notes
- presenter: shares findings
```

Some explanation

◆ Definition: Inclusive design means activities are accessible across prior skill, language confidence, and learning preference.

■ Note: A learner can contribute meaningfully without typing every command.

✦ Lightbulb Takeaway: Inclusion improves technical quality because more perspectives examine the evidence.

Accessibility checklist

1. explain each command purpose aloud
2. provide printed workflow cards
3. display outputs with readable contrast and zoom
4. avoid jargon without definition
5. offer sentence starters for reflections

Reflection scaffolds

- beginner: "One thing that changed was ____."
- intermediate: "The output changed because ____ evidence shows ____."
- advanced: "A limitation of this result is ____, so I would test ____ next."

Inclusive assessment options

- written response
- verbal explanation
- visual comparison board
- paired explanation interview

After you interact: What you learned

- You learned how role-based participation lets learners contribute meaningfully even when coding confidence differs.
- You learned how accessibility supports, like clear contrast and sentence starters, reduce cognitive overload during technical tasks.
- You learned how inclusive assessment formats capture understanding in more than one communication style.

Reflection Questions

- Which role assignment pattern in this chapter would best support your current group mix?
- Which accessibility checklist item is easiest to implement immediately, and which needs planning?
- Which assessment format would reveal understanding for a learner who struggles with written reflection?

What to Try Next in This Chapter

- Redesign one existing activity with explicit operator, observer, analyst, and presenter roles.
- Run the redesigned activity with a partner group and compare whether role clarity improved contribution balance.

Chapter 40: Continuous Improvement Plan

Intro into this chapter

This chapter helps you evolve the curriculum over multiple cohorts.

What you are going to use

- retrospective notes
- quality metrics
- student feedback loops

What you will learn in this chapter

- how to improve lesson quality each cycle
- how to prioritize high-value changes
- how to track impact over time

The work, clearly laid out

1. collect post-session evidence
2. identify bottlenecks and confusion points
3. prioritize one improvement per cycle
4. test improvement in next delivery
5. compare outcomes and iterate

Examples of what you might see

```
Cycle note:
Issue: students confused by prompt fairness.
Change: added fixed-prompt worksheet.
Result: stronger evidence quality in reflections.
```

Some explanation

◆ Definition: Retrospective means structured review of what worked, what failed, and what to improve next.

■ **Note: Small iterative improvements compound quickly over a term.**

❖ **Lightbulb Takeaway: Treat teaching quality like model quality—measure, adjust, repeat.**

Improvement dashboard template

1. Session date:
2. Objective met (yes/no):
3. Evidence quality score (1-5):
4. Student confidence score (1-5):
5. Biggest friction point:
6. Improvement chosen:
7. Next session success signal:

Change prioritization rubric

- high impact, low effort: do immediately
- high impact, medium effort: schedule next cycle
- low impact, high effort: defer unless required

After you interact: What you learned

- You learned how to run a repeatable improvement cycle by linking observed friction to one concrete change.
- You learned how the prioritization rubric helps you choose high-impact adjustments instead of reacting to every issue at once.
- You learned how dashboard signals, such as evidence quality and confidence, show whether your changes actually worked.

Reflection Questions

- Which session metric showed the clearest improvement since you started teaching with Kairo?
- What one change to your delivery would most reduce repeat troubleshooting in future sessions?

- How will you decide when your improvement plan needs a full reset rather than a small adjustment?

What to Try Next in This Chapter

- Complete one improvement dashboard entry using data from your most recent session.
- Compare dashboard entries with a peer and agree on one shared high-impact, low-effort change to test next.

Chapter 41: Glossary (Terms and Parameters)

Intro into this chapter

This chapter is a complete meaning guide for technical terms and command parameters used across the whole book.

What you are going to use

- term definitions
- parameter definitions
- quick lookup while practicing

What you will learn in this chapter

- what each key concept means
- what each command parameter controls
- how to explain workflow language clearly

The work, clearly laid out

1. review unknown terms
2. map terms to chapters where used
3. review parameter meanings before reruns

Examples of what you might see

```
--epochs 1: one pass through training data
--device cpu: use CPU instead of GPU
```

Some explanation

✦ Lightbulb Takeaway: Shared vocabulary makes collaboration and teaching easier.

Core terms

- **Attention**: Mechanism that weighs prior tokens when predicting the next token.
- **Batch**: Group of sequences processed in one training step.
- **Checkpoint**: Saved model state such as `best.pt`.
- **Context**: Reference text supplied before generation.
- **Corpus**: Prepared text collection used for training.
- **Dataset**: Source text used for training or evaluation.
- **Epoch**: One full pass through the training data.
- **Grounding**: Anchoring output to supplied context.
- **Inference**: Running a trained model to generate output.
- **Loss**: Numerical error signal used in training/evaluation.
- **Model architecture**: Structural design (layers, heads, model width).
- **Next-token prediction**: Predicting the next token from prior tokens.
- **Perplexity**: Uncertainty measure derived from loss.
- **Prompt**: Input text to start generation.
- **Provenance**: Record of where source data came from and how it was prepared.
- **Rubric**: Structured scoring guide for evaluating learner work.
- **Capstone**: Final synthesis project that demonstrates end-to-end mastery.
- **Retraining**: Training again on new or changed data.
- **Token**: Unit of text processed by the model (byte-based in Kairo).
- **Validation**: Performance check on held-out data.

Parameter glossary (entire book)

- `--input_file`: Path to training or evaluation text file.
- `--out_dir`: Directory for outputs such as checkpoints and logs.
- `--epochs`: Number of full training passes through dataset.
- `--batch_size`: Number of training sequences per optimization step.
- `--seq_len`: Token window length used by training/generation pipeline.
- `--d_model`: Model hidden dimension (representation width).
- `--n_heads`: Number of attention heads per transformer block.
- `--n_layers`: Number of transformer layers.

- `--device`: Hardware target, commonly `cpu` or `cuda`.
- `--checkpoint`: Path to saved model checkpoint for loading.
- `--prompt`: Input seed text for generation.
- `--max_new_tokens`: Maximum number of generated tokens to append.
- `--temperature`: Sampling randomness control (higher is more random).
- `--top_k`: Restrict sampling to top-k probable next tokens.
- `--input_jsonl`: JSONL source for QA corpus conversion.
- `--output_file`: File path for generated corpus output.
- `--question`: User question string for QA mode.
- `--context`: Inline context text used to ground QA response.
- `--context_file`: Path to context text file for QA grounding.
- `--help`: Shows command usage and available options without running the full action.

After you interact: What you learned

- You learned how to decode core AI terms in this book so chapter instructions are easier to follow.
- You learned how parameter meanings, such as `--epochs` and `--temperature`, map directly to training and generation behavior.
- You learned how a shared glossary supports clearer explanations when teaching or collaborating.

Reflection Questions

- Which three glossary terms do you now use differently because their definitions are clearer?
- Which parameter from this chapter most changed how you interpret a training or generation result?
- Where in the book would adding a glossary callout help beginners connect concept to command faster?

What to Try Next in This Chapter

- Create a personal mini-glossary of ten terms you want to master and write one example for each.

- Trade mini-glossaries with a peer and compare which definitions felt easiest or hardest to apply in practice.

Chapter 42: Conclusion

Intro into this chapter

You have now completed a full beginner-to-practice AI journey. This conclusion helps you anchor what matters most so your confidence lasts beyond this book.

What you are going to use

- your completed chapter notes
- your saved command outputs
- your reflections from guided labs

What you will learn in this chapter

- what core skills you now have
- how to carry those skills into future projects
- how to continue learning without overwhelm

The work, clearly laid out

1. review your strongest evidence moments
2. identify one skill you can now explain clearly
3. choose one next project to continue your growth

Examples of what you might see

```
"I can now compare baseline and retrained output using evidence."
"I can explain why scope, data, and prompt design change model behavior."
```

Some explanation

❖ Lightbulb Takeaway: Real confidence is the ability to explain *why* something changed, not just to say that it changed.

You have practiced observation, comparison, and explanation repeatedly. That is the core of technical confidence. These habits will transfer to new tools, larger models, and more advanced AI systems.

After you interact: What you learned

- You learned how to summarize your growth by pointing to concrete outputs, comparisons, and reflections from across the book.
- You learned how baseline training, retraining, evaluation, and QA grounding connect into one complete workflow.
- You learned how to choose a realistic next project so your confidence continues through practice.

Reflection Questions

- Which chapter produced the strongest evidence that your reasoning skills improved, and what proves it?
- Which part of the end-to-end workflow still feels least stable for you in real use?
- What is your first follow-on project, and which two chapters will you revisit before starting it?

What to Try Next in This Chapter

- Write a one-page personal summary that links one skill, one evidence example, and one next-step project.
- Share your summary with a peer and compare which evidence each of you chose to represent progress.

Chapter 43: About the Author

Intro into this chapter

This chapter introduces the author and the teaching intent behind the *Tech I Can* series.

What you are going to use

- a short author profile
- the educational mission of this book
- guidance on how to use the material in schools

What you will learn in this chapter

- who wrote this guide
- why the content is structured for classrooms
- what values informed the book design

The work, clearly laid out

1. read the author profile
2. connect the mission to your own classroom goals
3. decide how to adapt the material for your learners

Examples of what you might see

```
Focus: practical AI literacy
Method: explain -> run -> observe -> reflect
Audience: beginners, teachers, and curious learners
```

Some explanation

❖ Lightbulb Takeaway: The best technical teaching makes people feel capable, not intimidated.

Paul McMurray is the founder of Tech I Can, and his focus is practical AI literacy for schools. He works to make technical ideas teachable through clear steps that students can run,

observe, and explain. His teaching philosophy is evidence-first and beginner-friendly: test one change at a time, show what happened, and say honestly what is still uncertain. Across the Tech I Can series, his classroom mission is to help teachers and learners build confidence without hype, so AI becomes a tool for thoughtful learning, not a mystery.

After you interact: What you learned

- You learned why the author frames AI learning as practical, evidence-based classroom work.
- You learned how the book's teaching sequence builds confidence by moving from explanation to action and reflection.
- You learned how to adapt the same method to your own learners while keeping clarity and honesty at the center.

Reflection Questions

- Which part of the teaching philosophy in this chapter most matches how you want to teach?
- Which part would you adjust for your own class age group or confidence level?
- How will you keep lessons practical and evidence-based when students ask for faster, less structured demos?

What to Try Next in This Chapter

- Write a short teaching intent statement for your next Kairo session using the values from this chapter.
- Compare teaching intent statements with a peer and note one phrasing choice that makes the classroom goal clearer.

Chapter 44: Key Words Index

Intro into this chapter

This chapter gives you a fast lookup list of important terms and where they appear in the book.

What you are going to use

- alphabetical term list
- page references
- glossary links for full definitions

What you will learn in this chapter

- how to locate core concepts quickly
- how to revise efficiently before teaching or presenting
- how to connect terms back to practical examples

The work, clearly laid out

1. choose a term you want to review
2. jump to the listed page(s)
3. revisit the chapter example and explanation

Examples of what you might see

```
Token -> pages 47, 53, 95, 111
Grounding -> pages 66, 98, 111
Prompt -> pages 45, 64, 85, 113
```

Some explanation

■ **Note: Page references are generated from the final printable layout.**

❖ **Lightbulb Takeaway: A strong index turns a good book into a useful working tool.**

Key words and page numbers

Use this index to jump quickly to where each term is taught.

Format: primary pages show the best starting points; full references show complete page ranges.

After you interact: What you learned

- You learned how to use the keyword index as a rapid route back to concepts you need before teaching or revision.
- You learned how index references and glossary entries work together to connect terms to practical chapter examples.
- You learned how planned index use reduces last-minute searching and improves lesson preparation quality.

Reflection Questions

- Which indexed term did you locate fastest, and what chapter example made it clear?
- Which important term still needs a better cross-reference for your own revision workflow?
- How would you organize three priority terms before a live teaching session?

What to Try Next in This Chapter

- Build a "top ten before class" keyword list using the index and note the page for each.
- With a peer, compare your top-ten lists and identify which missing terms should be added for stronger preparation.